

BPA : Backward Propagation Attack

Transfer Attacks in Face Recognition

Om Singh Rawat (IIT D)

June 12, 2026

Attacks Considered

- PGD (baseline)
- MI-FGSM (momentum-based)
- TI-FGSM (translation-invariant)
- SI-NI-FGSM (scale-invariant Nesterov)
- MI-ADMIX-DI-TI (admixed input diversity)
- BPA-CNN (backward propagation adaptation)

Selecting BPA

Why BPA?

- ❑ Recent NeurIPS 2023 publication
- ❑ Highly effective transferability across CNN-based models
- ❑ Present in recent literature
- ❑ Not implemented in the intern baseline

Code at – [GitHub Repo](#)

BPA (Backward Propagation Attack)

Definition & Nature

- ❑ White-box transfer-based adversarial attack
- ❑ Gradient-based optimization method
- ❑ Adapted for black-box/pre-trained CNN models
- ❑ Based on NeurIPS 2023 paper implementation

Basic Idea

- ❑ Replaces sharp backward pass operations (ReLU, MaxPool) with smooth alternatives
- ❑ Smooths gradients at the input level to prevent overfitting to the surrogate model
- ❑ Counteracts ReLU binary gradient masking and MaxPool winner-take-all gradient concentration

BPA- Goal

- Operates in embedding space
- **Impersonation**: increase cosine similarity
- **Dodging**: decrease cosine similarity
- Optimizes embedding distance directly

Working of BPA attack

1. Initialize adversarial image as original input.
2. Define attack objective (based on attack type).
3. For T iterations:
 - Compute gradient of objective w.r.t input
 - Scale gradient and apply SiLU-derivative smoothing
 - Apply Gaussian spatial smoothing to the gradient
 - Normalize gradient and update accumulated gradient
 - Update image using sign of accumulated gradient
 - Project image into ϵ bounded region
 - Clip to valid pixel range
4. Return final adversarial image

Understanding BPA

BPA = Backward Propagation Attack

- Targets the sharp operations (ReLU, MaxPool) that cause gradient masking in CNNs.

SiLU-Derivative Scaling

- Replaces sharp ReLU gradients with smooth SiLU-inspired gradients.
- Counteracts binary gradient masking.

Gaussian Spatial Smoothing

- Replaces MaxPool's winner-take-all gradient concentration with a spatial Gaussian blur.
- Spreads the gradient information across adjacent pixels.

Momentum Update

- Like MI-FGSM, maintains accumulated gradient
- $g = decay * g + grad$
- $adv = adv + alpha * sign(g)$

Attack Implementation

```
ALL_ATTACKS = [  
    'PGD', 'MI_FGSM',  
    'TI_FGSM', 'SI_NI_FGSM',  
    'MI_ADMIX_DI_TI',  
    'BPA_CNN' # <-- added  
]
```

BPA-CNN function Pseudo Code

1. Initialize adversarial image
2. Compute embedding similarity & loss
3. Compute gradient
4. Apply SiLU-derivative smoothing
5. Apply Gaussian spatial smoothing
6. Normalize gradient & Update momentum
7. Project into ϵ -ball & Clip image
8. Repeat for NUM_ITER

```
if attack_name == "BPA_CNN":  
    return bpa_cnn(...)
```

BPA in Our Implementation

1. Initializes momentum variable **g**
2. Uses SiLU-derivative gradient scaling (**silu_deriv**) with Temperature = 3.0
3. Applies spatial Gaussian smoothing with Kernel = 5, Sigma = 1.0
4. Normalizes and updates accumulated gradient: **g = DECAY * g + grad**
5. Standard ϵ -projection via clipping ($\epsilon = 0.062$)
6. Fixed number of iterations (**NUM_ITER = 5**, **DECAY = 1.0**)

Full implementation → [GitHub Repo](#)

Evaluation Pipeline

Pipeline Flow

1. Load adversarial image
2. Load victim model
3. Compute embedding
4. Load target image
5. Compute target embedding
6. Compute cosine similarity
7. Store CSV row

Output Files

- `subset_raw_similarities_long.csv`
- Processed by `build_subset_baselines.py`
- Aggregated into `final_check/` folders

CSV Schema

```
row_id,  
attacker_model,  
img1,  
img2,  
dataset,  
attack_type,  
victim_model,  
attack_method,  
variant,  
similarity
```

Final Result

🏆 Attack Performance Comparison

Rank	Attack Method	Overall Breach Rate	Dodging Success	Impersonation Success	Impact Mean
1	BPA_CNN	29.58%	39.17%	20.00%	0.174
2	SI_NI_FGSM	28.33%	34.58%	22.08%	0.170
3	MI_FGSM	25.62%	32.92%	18.33%	0.158
4	MI_ADMIX_DI_TI	22.92%	27.08%	18.75%	0.143
5	TI_FGSM	20.63%	24.58%	16.67%	0.127
6	PGD	16.67%	20.83%	12.50%	0.096

- **BPA-CNN** outperformed the best baseline (SI-NI-FGSM) in overall transferability by +1.25%.
- It achieved the highest Dodging success rate (+4.59% over SI_NI_FGSM).

Why so high?

- BPA was explicitly designed to bypass gradient masking and concentration inherent in CNNs.
- Since all baseline models (VGG, ResNet-based ArcFace) are CNNs, the input-level smoothing heavily disrupted their shared structural vulnerabilities, leading to massive transferability

Thank You

Om Singh Rawat - IITD

Full Implementation – [GitHub Repo](#)